CausalFlow: Generating Counterfactual Training Data for LLM Agent Post-Training

Anonymous Authors[1]

Abstract

Training data for agentic capabilities is scarce and expensive to collect, as it requires multi-step reasoning traces with tool use that are difficult to generate synthetically. We introduce CausalFlow, a pipeline that automatically generates high-quality (failed_trace, repaired_trace) pairs from LLM agent failures for post-training. Our key insight is that counterfactual repairs provide contrastive supervision—models can learn both what went wrong and how to fix it from paired examples. CausalFlow applies causal inference to identify which steps caused failures, generates minimal repairs through counterfactual intervention, and validates data quality through multi-agent critique. The resulting training pairs include step-level causal annotations, minimal edit distances, and skill taxonomy labels for curriculum learning. Experiments across four benchmarks—GSM8K, MBPP, SealQA, and MedBrowseComp—demonstrate that CausalFlow generates 555 validated training pairs with average minimality scores of 0.79–0.87 and consensus scores of 0.85–0.91, establishing a scalable approach to synthesizing training data for agent post-training.

1. Introduction

LLM-powered agents have emerged as a promising paradigm for autonomous problem-solving, combining language model reasoning with tool use, memory access, and environmental interaction (Yao et al., 2023; Schick et al., 2023; Nakano et al., 2021). However, training agents to perform complex multi-step tasks remains challenging due to a fundamental data scarcity problem: high-quality training data for agentic capabilities is expensive to collect and difficult to generate synthetically.

Current approaches to agent training face three key limitations. First, agentic training data requires multi-step reasoning traces with tool use, which are expensive to manually annotate at scale. Second, typical synthetic generation methods lack error correction signals—they produce either successful traces or failed traces, but not paired examples showing what went wrong and how to fix it. Third, existing data lacks step-level supervision; annotations indicate only whether the final outcome was correct, not which intermediate steps caused failures.

We propose a novel perspective: agent failures are a rich source of training data, not just evaluation artifacts. Every failed execution, when paired with a targeted repair, becomes a contrastive training example. The key is identifying which steps to repair and generating minimal corrections that teach precise fixes rather than wholesale rewrites.

We introduce CausalFlow, a pipeline for generating high-quality (failed_trace, repaired_trace) pairs from agent failures. Given a failed execution, CausalFlow:

1. Constructs a causal graph from explicit step dependencies
2. Computes Causal Responsibility Scores (CRS) to identify which steps caused the failure
3. Generates minimal counterfactual repairs that fix only the causal steps
4. Validates data quality through multi-agent critique

Our key insight is that counterfactual repairs provide contrastive supervision: models see both the incorrect reasoning and the corrected version for the same problem. This paired structure, combined with step-level causal annotations, enables more targeted learning than training on successful traces alone.

[1]Anonymous Institution, Anonymous City, Anonymous Region, Anonymous Country. Correspondence to: Anonymous Author <anon.email@domain.com>.

Preliminary work. Under review by the International Conference on Machine Learning (ICML). Do not distribute.

We evaluate CausalFlow on four diverse benchmarks spanning mathematical reasoning (GSM8K), code generation (MBPP), question answering (SealQA Hard), and medical web browsing (MedBrowseComp). Our pipeline generates 555 validated training pairs with high minimality scores (0.79–0.87) and strong consensus validation (0.85–0.91). We also organize generated data by a skill taxonomy of 12–16 categories per domain, enabling curriculum-based training.

Contributions. Our main contributions are:

- A pipeline for automatically generating contrastive (failed, repaired) training pairs from agent execution failures
- Step-level causal annotations via Causal Responsibility Scores (CRS) that identify exactly which steps to learn from
- Quality-controlled data generation through minimality constraints and multi-agent consensus validation
- Extensive experiments generating 555 training pairs across four benchmarks, with analysis of data quality metrics and skill taxonomies

2. Related Work

Synthetic Data for LLM Training. Generating synthetic training data has become essential for LLM development. Self-Instruct (Wang et al., 2023) generates instruction-following data from seed examples. RLHF pipelines (Ouyang et al., 2022) create preference data through human feedback. Rejection sampling approaches (Yuan et al., 2023) filter model outputs for quality. Constitutional AI (Bai et al., 2022) uses self-critique to generate preference pairs. However, these methods focus on single-turn or simple multi-turn settings, not complex agentic traces with tool use. CausalFlow addresses this gap by generating contrastive training pairs specifically from agent execution failures.

Process Supervision. Recent work has shown that step-level supervision improves reasoning capabilities. Process reward models (Lightman et al., 2024) train verifiers on human-annotated step correctness. Outcome vs. process supervision (Uesato et al., 2022) demonstrates benefits of intermediate feedback. However, obtaining step-level labels is expensive. CausalFlow automatically generates step-level annotations through causal attribution, identifying which steps caused failures without human labeling.

Causal Inference in Machine Learning. Causal inference provides tools for understanding interventional effects and counterfactual reasoning (Pearl, 2009; Peters et al., 2017). Applications to ML interpretability include causal feature attribution (Schwab & Karlen, 2019), causal concept explanations (Goyal et al., 2019), and causal tracing in transformers (Meng et al., 2022). Our work extends these ideas to the structured domain of agent execution traces, where explicit step dependencies enable direct causal graph construction. We use causal attribution not for explanation but for data generation: identifying which steps to repair to create training pairs.

Error Analysis and Self-Correction. Prior work has analyzed LLM errors through behavioral testing (Ribeiro et al., 2020) or chain-of-thought analysis (Wei et al., 2022). Self-debugging approaches (Chen et al., 2024) have models fix their own errors but lack principled identification of which steps to fix. Our CRS-based approach provides formal grounding for identifying causal steps, enabling targeted repair generation rather than wholesale trace rewriting.

Multi-Agent Critique Systems. Using multiple LLM agents for verification and critique has shown promise for improving reliability (Du et al., 2023; Liang et al., 2023). We adapt multi-agent critique specifically for validating training data quality, using ensemble agreement to filter low-quality repairs and ensure generated pairs meet quality thresholds.

3. Preliminaries

3.1. Agent Execution Traces

We model an LLM agent execution as a sequence of typed steps $\tau = (s_1, s_2, \ldots, s_n)$, where each step s_i has:

- A type $t_i \in \mathcal{T}$, where $\mathcal{T} = \{\text{reasoning}, \text{tool\_call}, \text{tool\_response}, \text{llm\_response}, \text{memory\_access}, \text{environment\_action}, \text{environment\_observation}, \text{final\_answer}\}$
- Type-specific content (e.g., reasoning text, tool name and arguments, or observation data)
- A set of dependencies $D_i \subseteq \{1, \ldots, i-1\}$ indicating which prior steps s_i directly depends on

The final step is always of type final_answer and represents the agent's output. We denote the agent's final answer as $a(\tau)$ and the ground truth as a^*. An execution is successful if $a(\tau) = a^*$ and failed otherwise.

3.2. Causal Graphs from Traces

The dependencies D_i naturally induce a directed acyclic graph (DAG) $G = (V, E)$ where $V = \{s_1, \ldots, s_n\}$ and $(s_j, s_i) \in E$ iff $j \in D_i$. This causal graph represents the causal structure of the execution: s_i causally depends on its ancestors in G.

Definition 3.1 (Causal Ancestors). For step s_i, the causal ancestors $\text{Anc}(s_i)$ are all steps reachable by following edges backwards from s_i. These are the steps that could potentially influence s_i.

Definition 3.2 (Critical Steps). A step s_i is critical if it lies on every path from the first step to the final answer. Failures in critical steps necessarily affect the outcome.

4. The CausalFlow Pipeline

We now present the CausalFlow pipeline for generating training data from agent failures. The pipeline has four components: capturing agent behavior, identifying teachable moments through causal attribution, generating contrastive training pairs, and validating data quality.

4.1. Capturing Agent Behavior

The first stage captures detailed execution traces from agent runs. These traces form the raw material from which training data is generated. Failed traces (where $a(\tau) \neq a^*$) are candidates for training pair generation, as each failure represents an opportunity to create a contrastive example.

4.2. Identifying Teachable Moments

Not every step in a failed trace is worth learning from. Our goal is to identify which steps are causally responsible for the failure—these are the "teachable moments" where targeted corrections provide the most training signal. We adopt an interventionist approach inspired by actual causation (Halpern & Pearl, 2005): a step is causally responsible if intervening on it would change the outcome.

Definition 4.1 (Causal Responsibility Score). For a failed trace τ with outcome $a(\tau) \neq a^*$, the Causal Responsibility Score of step s_i is:

$$\text{CRS}(s_i) = \mathbb{I}\left[a(\tau_{s_i \leftarrow s_i'}) = a^*\right] \tag{1}$$

where $\tau_{s_i \leftarrow s_i'}$ denotes the trace obtained by replacing s_i with an intervened version s_i' and re-executing all subsequent steps, and $\mathbb{I}[\cdot]$ is the indicator function.

Intuitively, $\text{CRS}(s_i) = 1$ if fixing step s_i (while propagating effects forward) leads to a successful outcome.

Algorithm 1 Causal Responsibility Scoring

```
Input: Failed trace τ = (s_1, ..., s_n), gold answer a*
Output: CRS scores {CRS(s_i)}_{i=1}^{n-1}
for i = 1 to n − 1 do
  s_i' ← GenerateIntervention(s_i, τ_{<i}, feedback)
  if s_i' = s_i then
    CRS(s_i) ← 0 {No change proposed}
  else
    τ' ← ReExecute(τ, i, s_i')
    CRS(s_i) ← 𝕀[a(τ') = a*]
  end if
end for
return {CRS(s_i)}
```

This provides a binary measure of causal responsibility.

Computing Interventions. To compute s_i', we prompt an LLM with the original step s_i, the trace context up to s_i, and feedback about the execution failure. The LLM generates a "corrected" version of the step. For reasoning steps, this corrects logical errors; for tool calls, this fixes incorrect arguments or tool selection.

Re-execution. After intervention on s_i, we re-execute the trace from step $i + 1$ onwards. This can be done either:

- Deterministically: If a domain-specific executor is available (e.g., Python interpreter for code), we re-run the actual computation
- Predictively: We use an LLM to predict whether the intervened trace would succeed, based on the corrected step and original context

Algorithm 1 summarizes the CRS computation.

4.3. Generating Contrastive Training Pairs

For steps with $\text{CRS}(s_i) = 1$ (i.e., causally responsible steps), we generate counterfactual repairs—minimal modifications that, combined with the original failed trace, form contrastive training pairs.

Minimality for Training Quality. A key desideratum is that repairs should be minimal: they should fix only the identified error without introducing unnecessary changes. Minimal repairs ensure models learn targeted corrections rather than wholesale rewrites, making the training signal more precise. We define minimality as:

$$\text{Minimality}(s_i, s_i') = \frac{|\text{tokens}(s_i) \cap \text{tokens}(s_i')|}{|\text{tokens}(s_i) \cup \text{tokens}(s_i')|} \tag{2}$$

measuring token overlap between original and repaired steps. Higher scores indicate smaller edits.

Repair Generation. We generate multiple repair proposals per causal step using temperature sampling, then rank by minimality. The repair prompt emphasizes:

1. Fixing the logical error in the specific step
2. Not using the gold answer directly
3. Making the smallest possible change

Each repair is validated by re-execution (deterministic) or outcome prediction (LLM-based) to confirm it leads to success. Successfully validated pairs are added to the training dataset.

4.4. Validating Training Data Quality

To ensure high-quality training data, we employ a multi-agent critique system. Single-LLM repairs may introduce new errors or produce low-quality corrections; ensemble agreement filters out problematic training pairs.

Our system uses three agents:

- Agent A (Proposer): The repair generator that produces training pair candidates
- Agent B (Critic 1): Reviews the repair for correctness and quality
- Agent C (Meta-Critic): Reviews both the repair and Agent B's critique

Each critic agent outputs:

- Agreement: AGREE, DISAGREE, or PARTIAL
- Confidence: $[0, 1]$
- Reasoning about repair quality

We compute a consensus score as a weighted average:

$$\text{Consensus}(s_i) = \frac{1}{3}\left(\text{CRS}(s_i) + \sum_{j \in \{B,C\}} c_j \cdot a_j\right) \quad (3)$$

where $a_j \in \{0, 0.5, 1\}$ encodes agreement and c_j is confidence. Training pairs with $\text{Consensus}(s_i) \geq 0.5$ are included in the final dataset.

4.5. Training Data Properties

The resulting training data has several properties valuable for agent post-training:

Contrastive Pairs. Each training example contains both the failed trace and the repaired trace for the same problem. This contrastive structure enables models to learn from explicit comparisons between incorrect and correct reasoning.

Step-Level Annotations. CRS provides binary labels indicating which steps caused the failure. This enables process supervision—models can receive feedback at individual steps, not just final outcomes.

Minimal Edits. By ranking repairs by minimality, training pairs teach targeted corrections. Models learn precise fixes rather than learning to completely rewrite traces.

Skill Taxonomy. We cluster causal steps by error patterns to organize training data into skill categories (e.g., arithmetic errors, loop logic bugs, query formulation issues). This enables curriculum learning where models can be trained on specific skill deficits.

5. Experiments

We evaluate CausalFlow on four diverse benchmarks to answer:

- Q1 How much training data can CausalFlow generate from agent failures?
- Q2 What is the quality of generated training pairs (minimality, consensus)?
- Q3 What skill categories does the generated data cover?

5.1. Experimental Setup

Benchmarks. We evaluate on four benchmarks spanning different domains:

- GSM8K (Cobbe et al., 2021): Grade school math word problems requiring multi-step arithmetic reasoning
- MBPP (Austin et al., 2021): Python programming problems from the Mostly Basic Programming Problems benchmark
- SealQA Hard (Pham et al., 2025): Complex question answering requiring web search and synthesis

- MedBrowseComp (Chen et al., 2025): Medical question answering with web browsing capabilities

Agent Setup. For each benchmark, we run an appropriate LLM agent:

- GSM8K: Gemini 2.0 with calculator tool
- MBPP: GPT-5 Chat with Python execution
- SealQA Hard: Gemini 3 Flash with web search
- MedBrowseComp: Gemini 3 Flash with web browsing

Metrics. We report:

- Training Pair Generation Rate: Fraction of failed executions converted into validated training pairs
- Minimality Score: Average token overlap between original and repaired steps (higher = smaller edits)
- Consensus Score: Average multi-agent agreement on repair quality
- Skill Groups: Number of distinct skill categories in generated data

5.2. Training Data Generation Results

Table 1 presents training data generation results across all benchmarks.

Table 1. Training data generation results. CausalFlow generates 555 validated training pairs across four benchmarks with high quality scores.

Benchmark	Failed	Pairs	Rate	Min.	Cons.
GSM8K	330	173	52.4%	0.87	0.91
MBPP	488	201	41.2%	0.82	0.88
SealQA Hard	146	32	21.9%	0.79	0.85
MedBrowseComp	335	149	44.5%	0.84	0.89
Total	1299	555	42.7%	0.83	0.88

CausalFlow achieves its highest generation rate on GSM8K (52.4%), where errors tend to be localized arithmetic or reasoning mistakes amenable to minimal repair. GSM8K also achieves the highest quality scores (0.87 minimality, 0.91 consensus). The lowest rate is on SealQA Hard (21.9%), where failures often involve information retrieval gaps that cannot be fixed by modifying reasoning steps alone.

Training Pair Characteristics. Generated training pairs exhibit several patterns:

- Arithmetic corrections (GSM8K): Pairs showing incorrect → correct calculation steps
- Logic bug fixes (MBPP): Pairs demonstrating off-by-one error corrections, boundary condition handling
- Query refinement (SealQA, MedBrowseComp): Pairs showing improved search query formulation
- Reasoning corrections: Pairs fixing incorrect inferences or missed constraints

5.3. Skill Taxonomy for Curriculum Learning

CausalFlow's causal attributions reveal skill categories that organize generated training data. We cluster causal steps by their error patterns to create a skill taxonomy, enabling curriculum-based training (Table 2). GSM8K has the most diverse categories (16), including arithmetic operations, unit conversions, multi-step planning, and constraint tracking. MBPP clusters into 12 categories covering loop logic, string manipulation, and edge case handling. This taxonomy enables targeted training: practitioners can select training pairs for specific skills their models struggle with, or construct curricula progressing from simpler to more complex skill categories.

5.4. Qualitative Examples

Example 1: Multi-Step Reasoning with Implicit Constraints. Consider a problem: "A store sells notebooks for $4 each. Maria buys some notebooks and pays with a $20 bill. She receives $8 in change. She then gives half her notebooks to her brother. How many notebooks does Maria have now?"

The agent trace shows:

1. Reasoning: "Maria paid $20 and got $8 change, so she spent $20 - $8 = $12"
2. Tool call: calculator(20 - 8) = 12
3. Reasoning: "Each notebook costs $4, so she bought $12 / $4 = 3 notebooks"
4. Tool call: calculator(12 / 4) = 3
5. Reasoning: "She gives half to her brother, so she keeps 3 / 2 = 1" [ERROR: integer division without considering whole objects]
6. Final answer: 1 [INCORRECT—should be 2]

CausalFlow identifies Step 5 as causal (CRS = 1) and generates the repair: "She bought 3 notebooks. Giving half means 3 / 2 = 1.5, but since notebooks are whole items, she gives 1 to her brother and keeps 2."

Training Value. This contrastive pair teaches the model to recognize implicit constraints (whole objects cannot be divided) when performing division in word problems. The minimal edit targets specifically the step where the constraint was missed, rather than rewriting the entire trace.

Example 2: MBPP Logic Bug. A function to check if a number is prime incorrectly returns True for 1:

```
def is_prime(n):
    for i in range(2, n):
        if n % i == 0:
            return False
    return True  # Bug: n=1 returns True
```

CausalFlow traces the failure to the missing base case and generates a minimal repair adding if n <= 1: return False.

Training Value. This pair teaches edge case handling: the original trace shows the common pattern of forgetting boundary conditions, and the repair demonstrates the minimal fix. The contrastive structure helps models learn to check base cases when implementing mathematical predicates.

5.5. Ablation Studies

Multi-Agent Critique for Data Quality. Removing multi-agent critique and relying solely on CRS decreases average consensus scores by 23%. The critique system filters low-quality repairs where the LLM proposes changes that technically fix the issue but introduce other problems or provide poor training signal.

Minimality Constraint. Without minimality ranking, repairs average 3.2× more token changes. While these "heavy" repairs still lead to correct answers, they provide weaker training signal—models learn to rewrite traces rather than make targeted fixes. The minimality constraint ensures training pairs teach precise corrections.

6. Discussion

Key Findings. Our experiments reveal several insights about generating training data from agent failures:

Table 2. Skill taxonomy for generated training data. Numbers indicate distinct categories for organizing training pairs by skill deficit.

Benchmark	Skills	Examples
GSM8K	16	arithmetic, unit conv.
MBPP	12	loop logic, strings
SealQA Hard	4	query, synthesis
MedBrowseComp	6	med. reasoning

1. Failures are often localized: In most cases, 1–3 steps are causally responsible, enabling targeted training pairs rather than wholesale trace rewrites
2. Generation rates vary by domain: Closed-domain tasks (GSM8K, MBPP) yield higher-quality training data than open-domain retrieval tasks
3. Skill taxonomies emerge: Causal analysis naturally organizes training data into interpretable skill categories for curriculum learning

Implications for Agent Training. The contrastive training pairs generated by CausalFlow enable several training approaches:

- Process supervision: Step-level CRS annotations provide supervision signal at intermediate steps, not just final outcomes
- Curriculum learning: Skill taxonomies enable training curricula that target specific capability gaps
- Contrastive learning: (Failed, repaired) pairs can train models to distinguish correct from incorrect reasoning
- Self-improvement: Models can be trained on repairs of their own failures, creating a closed-loop improvement cycle

Limitations. CausalFlow has several limitations:

- Repair quality: LLM-generated repairs may not always represent optimal fixes; multi-agent critique mitigates but does not eliminate this
- Generation costs: Computing CRS requires rerunning (parts of) traces, which can be expensive for complex agents
- Retrieval failures: When failures stem from information not being retrievable (vs. reasoning errors), useful training pairs cannot be generated
- Coverage: Not all agent failures can be converted to training pairs; some require architectural changes rather than step repairs

Future Work. Promising directions include: (1) training agents on CausalFlow-generated data to measure downstream improvement, (2) active learning to prioritize failure cases most valuable for training, (3) extending to multi-agent systems where failures involve coordination, and (4) learning to predict CRS without re-execution to reduce generation costs.

7. Conclusion

We presented CausalFlow, a pipeline for generating high-quality contrastive training data from LLM agent failures. By applying causal inference to identify which steps caused failures and generating minimal counterfactual repairs, CausalFlow transforms agent failures into (failed_trace, repaired_trace) training pairs. Our multi-agent critique system ensures data quality through ensemble validation. Experiments on four benchmarks demonstrate that CausalFlow generates 555 validated training pairs with high minimality (0.79–0.87) and consensus (0.85–0.91) scores. The generated data includes step-level causal annotations and skill taxonomy labels, enabling process supervision and curriculum learning. We believe automated generation of contrastive training data from failures is a promising direction for scalable agent post-training.

Impact Statement

This paper presents work whose goal is to advance the field of Machine Learning, specifically in generating training data for improving LLM agent capabilities. CausalFlow enables practitioners to automatically generate high-quality training data from agent failures, potentially reducing the cost and effort required to improve agent systems.

The generated training data could be used to make agents more capable, which has both positive applications (more reliable AI assistants) and potential risks (more capable systems may be misused). We believe the benefits of improved agent training outweigh these risks, as the same training improvements could be achieved through more expensive manual annotation. Our work reduces costs but does not enable fundamentally new capabilities.

References

Austin, J., Odena, A., Nye, M., Bosma, M., Michalewski, H., Dohan, D., Jiang, E., Cai, C., Terry, M., Le, Q., et al. Program synthesis with large language models. arXiv preprint arXiv:2108.07732, 2021.

Bai, Y., Kadavath, S., Kundu, S., Askell, A., Kernion, J., Jones, A., Chen, A., Goldie, A., Mirhoseini, A., McKinnon, C., et al. Constitutional AI: Harmlessness from AI feedback. arXiv preprint arXiv:2212.08073, 2022.

Chen, S., Moreira, P., Xiao, Y., Schmidgall, S., Warner, J., Aerts, H., Hartvigsen, T., Gallifant, J., and Bitterman, D. S. MedBrowseComp: Benchmarking medical deep research and computer use. arXiv preprint arXiv:2505.14963, 2025.

Chen, X., Lin, M., Schärli, N., and Zhou, D. Teaching large language models to self-debug. International Conference on Learning Representations, 2024.

Cobbe, K., Kosaraju, V., Bavarian, M., Chen, M., Jun, H., Kaiser, L., Plappert, M., Tworek, J., Hilton, J., Nakano, R., et al. Training verifiers to solve math word problems. arXiv preprint arXiv:2110.14168, 2021.

Du, Y., Li, S., Torralba, A., Tenenbaum, J. B., and Mordatch, I. Improving factuality and reasoning in language models through multiagent debate. arXiv preprint arXiv:2305.14325, 2023.

Goyal, Y., Feder, A., Shalit, U., and Kim, B. Explaining classifiers with causal concept effect (CaCE). In arXiv preprint arXiv:1907.07165, 2019.

Halpern, J. Y. and Pearl, J. Causes and explanations: A structural-model approach. part I: Causes. The British Journal for the Philosophy of Science, 56(4): 843–887, 2005.

Liang, T., He, Z., Jiao, W., Wang, X., Wang, Y., Wang, R., Yang, Y., Tu, Z., and Shi, S. Encouraging divergent thinking in large language models through multi-agent debate. arXiv preprint arXiv:2305.19118, 2023.

Lightman, H., Kosaraju, V., Burda, Y., Edwards, H., Baker, B., Lee, T., Leike, J., Schulman, J., Sutskever, I., and Cobbe, K. Let's verify step by step. International Conference on Learning Representations, 2024.

Meng, K., Bau, D., Andonian, A., and Belinkov, Y. Locating and editing factual associations in GPT. In Advances in Neural Information Processing Systems, 2022.

Nakano, R., Hilton, J., Balaji, S., Wu, J., Ouyang, L., Kim, C., Hesse, C., Jain, S., Kosaraju, V., Saunders, W., et al. WebGPT: Browser-assisted question-answering with human feedback. arXiv preprint arXiv:2112.09332, 2021.

Ouyang, L., Wu, J., Jiang, X., Almeida, D., Wainwright, C., Mishkin, P., Zhang, C., Agarwal, S., Slama, K., Ray, A., et al. Training language models to follow instructions with human feedback. Advances in Neural Information Processing Systems, 35:27730–27744, 2022.

Pearl, J. Causality: Models, Reasoning, and Inference. Cambridge University Press, 2nd edition, 2009.

Peters, J., Janzing, D., and Schölkopf, B. Elements of Causal Inference: Foundations and Learning Algorithms. MIT Press, 2017.

Pham, T., Nguyen, N., Zunjare, P., Chen, W., Tseng, Y.-M., and Vu, T. SealQA: Raising the bar for reasoning in search-augmented language models. arXiv preprint arXiv:2506.01062, 2025.

Ribeiro, M. T., Wu, T., Guestrin, C., and Singh, S. Beyond accuracy: Behavioral testing of NLP models with CheckList. In Association for Computational Linguistics, 2020.

Schick, T., Dwivedi-Yu, J., Dessì, R., Raileanu, R., Lomeli, M., Hambro, E., Zettlemoyer, L., Cancedda, N., and Scialom, T. Toolformer: Language models can teach themselves to use tools. In Advances in Neural Information Processing Systems, volume 36, 2023.

Schwab, P. and Karlen, W. CXPlain: Causal explanations for model interpretation under uncertainty. In Advances in Neural Information Processing Systems, 2019.

Uesato, J., Kushman, N., Kumar, R., Song, F., Siegel, N., Wang, L., Creswell, A., Irving, G., and Higgins, I. Solving math word problems with process-and outcome-based feedback. arXiv preprint arXiv:2211.14275, 2022.

Wang, Y., Kordi, Y., Mishra, S., Liu, A., Smith, N. A., Khashabi, D., and Hajishirzi, H. Self-instruct: Aligning language models with self-generated instructions. Association for Computational Linguistics, 2023.

Wei, J., Wang, X., Schuurmans, D., Bosma, M., Ichter, B., Xia, F., Chi, E., Le, Q., and Zhou, D. Chain-of-thought prompting elicits reasoning in large language models. Advances in Neural Information Processing Systems, 2022.

Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., and Cao, Y. ReAct: Synergizing reasoning and acting in language models. International Conference on Learning Representations, 2023.

Yuan, Z., Yuan, H., Li, C., Dong, G., Lu, K., Tan, C., Zhou, C., and Zhou, J. Scaling relationship on learning mathematical reasoning with large language models. arXiv preprint arXiv:2308.01825, 2023.

A. Implementation Details

A.1. Trace Logger

The TraceLogger captures agent execution through typed step logging:

```
class StepType(Enum):
    REASONING = "reasoning"
    TOOL_CALL = "tool_call"
    TOOL_RESPONSE = "tool_response"
    LLM_RESPONSE = "llm_response"
    MEMORY_ACCESS = "memory_access"
    ENVIRONMENT_ACTION = "environment_action"
    ENVIRONMENT_OBSERVATION = "environment_observation"
    FINAL_ANSWER = "final_answer"
```

Each step records its type, content, and explicit dependencies on prior steps.

A.2. Causal Graph Construction

The causal graph is constructed directly from step dependencies using NetworkX:

```
def _build_graph(self):
    for step in self.trace.steps:
        self.graph.add_node(step.step_id, step_type=step.step_type.value)
    for step in self.trace.steps:
        for dep_id in step.dependencies:
            self.graph.add_edge(dep_id, step.step_id)
```

A.3. Intervention Prompts

The intervention prompt for CRS computation follows this template:

```
System: You are an expert at debugging agent reasoning steps.
Given a step that contributed to a failed execution, suggest
a corrected version that would lead to success.

Context: [Previous steps in trace]
Failed Step: [Step content]
Feedback: [Execution failure message]

Generate a minimal correction to this step.
```

A.4. Training Pair Validation

Training pairs are validated through either deterministic re-execution (when an executor is available) or LLM-based outcome prediction:

```
def _evaluate_repair_success(self, step_id, repaired_step):
    if self.reexecutor is not None:
        # Deterministic validation
        new_trace = self.reexecutor.execute(repaired_step)
        return new_trace.outcome == "success"
    else:
        # LLM prediction
        return self._predict_outcome(repaired_step)
```

Only pairs where the repair leads to successful execution are included in the final training dataset.

B. Extended Results

B.1. Per-Model Training Data Generation

Table 3. Training data generation broken down by base model used for agent execution.

Benchmark	Model	Failed	Pairs	Rate	Min.
GSM8K	Gemini 2.0	330	173	52.4%	0.87
MBPP	GPT-5 Chat	488	201	41.2%	0.82
SealQA Hard	Gemini 3 Flash	146	32	21.9%	0.79
MedBrowseComp	Gemini 3 Flash	335	149	44.5%	0.84

B.2. Skill Taxonomy Details

The following skill categories organize generated training data for curriculum learning:

GSM8K Categories (16): Basic arithmetic, multi-digit multiplication, division with remainders, unit conversion, percentage calculation, ratio and proportion, multi-step planning, constraint satisfaction, word problem parsing, variable tracking, comparison operations, time calculations, money calculations, distance/rate/time, combinatorics basics, and equation setup.

MBPP Categories (12): Loop construction, string manipulation, list operations, dictionary handling, recursion, edge case handling, type conversion, mathematical operations, conditional logic, input validation, output formatting, and algorithm selection.

SealQA Hard Categories (4): Query formulation, information retrieval, multi-source synthesis, and answer extraction.

MedBrowseComp Categories (6): Medical terminology understanding, symptom-condition mapping, treatment reasoning, drug interaction awareness, medical guideline interpretation, and clinical decision support.

Using the Taxonomy. Practitioners can use these categories to: (1) identify which skills their models struggle with, (2) select training pairs targeting specific deficits, and (3) construct curricula that progress from simpler to more complex skills within each domain.